

EVENT MANAGEMENT SYSTEM

Project Report

A console-based Java application for managing campus events, student registrations, event capacity, and operational reports.

Student	Utkarsh
Course / Context	Java — Object-Oriented Programming
Institution	VIT Bhopal University
Technology	Java, OOP, Collections, Exception Handling
Application Style	Menu-driven command-line application
Build Status	Compiles cleanly • all 8 TestRunner scenarios PASS

Table of Contents

1. Introduction.....	3
2. Problem Statement.....	3
3. Objectives.....	4
4. Functional Requirements.....	4
5. Non-Functional Requirements.....	5
6. System Architecture.....	6
7. Design Diagrams.....	8
7.1 Use Case Diagram.....	8
7.2 Process Flow / Workflow Diagram.....	9
7.3 Sequence Diagram — Student Registration.....	10
7.4 Class / Component Diagram.....	11
7.5 Storage Design.....	11
8. Design Decisions & Rationale.....	13
9. Implementation Details.....	13
10. Screenshots / Results.....	16
11. Testing Approach.....	18
12. Challenges Faced.....	19
13. Learnings & Key Takeaways.....	19
14. Future Enhancements.....	20
15. References.....	21
Appendix A — Project Structure & Requirement Traceability.....	22
Appendix B — Submission Readiness Notes.....	24

1. Introduction

The VIT Event Management System is a menu-driven Java application designed to organize common campus-event operations in a single console workflow. The system separates the responsibilities of students and organizers, and provides functionality for discovering events, registering for events, cancelling registrations, managing event records, and generating summary reports.

The implementation uses object-oriented Java design. Domain entities such as User, Student, Organizer, Event, Venue, and Registration are placed in a model package. EventManager, RegistrationManager, and ReportManager provide service-level operations. Custom checked exceptions represent important business-rule failures.

Runtime data is held in ArrayList collections rather than a persistent database. This keeps the project focused as a Java/OOP exercise while still demonstrating encapsulation, inheritance, abstraction, composition, collections, modularisation, and exception handling.

Project Snapshot

Aspect	Observed Implementation
Users	Student and Organizer roles
Core domain	Events, venues, registrations
Operations	Create, read/display, update, delete, search, register, cancel, report
Runtime storage	ArrayList<Event> and ArrayList<Registration>
Error handling	EventNotFoundException, EventFullException, AlreadyRegisteredException
Testing	Dedicated TestRunner.java with 8 test scenarios — all passing
Source organization	model / service / exception / util / test packages

2. Problem Statement

Campus events involve several recurring tasks: publishing event details, checking available capacity, accepting student registrations, preventing duplicate registrations, cancelling registrations, updating event information, and reviewing participation summaries. When these activities are handled manually or through disconnected records, it becomes harder to maintain consistent event capacity and registration information.

Problem statement: Develop a Java-based event management system that provides separate student and organizer workflows, maintains event and registration state during execution, enforces key registration rules, supports event CRUD operations, and produces useful event and registration reports.

The problem is deliberately scoped to the capabilities visible in the submitted code. The current version does not implement authentication, a database, a GUI, payment processing, email notifications, or network-based multi-user synchronization.

Target Users

- Students who want to browse, search, register for, view, and cancel event registrations.
- Organizers who need to create, update, delete, inspect, and report on events and registrations.

Project Scope

- In-scope: event lifecycle management, registration lifecycle, capacity control, duplicate-registration prevention, searching, reports, and console interaction.
- Out-of-scope: persistent storage, authentication/authorization, web/mobile interface, real-time distributed access, and external notification services.

3. Objectives

- Apply Java object-oriented programming concepts to a practical campus-management problem.
- Model real-world entities using classes with encapsulated state and meaningful methods.
- Provide at least three major functional modules with a logical user workflow.
- Implement CRUD-style event management for organizer operations.
- Implement registration validation using capacity and duplicate-registration rules.
- Use custom exception classes to communicate business-rule failures clearly.
- Use collections to maintain events and registrations during runtime.
- Provide summary reports for events and registrations.
- Separate model, service, exception, utility, and testing responsibilities into packages.
- Document the system using architecture, workflow, UML-style diagrams, implementation details, testing, and future enhancements.

These objectives align directly with the VITyarthi requirement to identify a meaningful problem, design a technical solution, implement it using subject concepts, and demonstrate understanding through documentation and evaluation.

4. Functional Requirements

ID	Module	Requirement	Input / Output
FR-01	Role selection	The application lets the user choose Student, Organizer, or Exit from the main menu.	Console choice → role-specific menu
FR-02	View events	Students and organizers can display all available events.	No input beyond menu choice
FR-03	Search event	Students can search event names using a case-insensitive keyword.	Keyword → matching events

ID	Module	Requirement	Input / Output
FR-04	Register	A student can register for a selected event when capacity exists and the student is not already registered.	Student + Event ID → Registration ID
FR-05	Cancel registration	A student can cancel an existing registration by registration ID.	Registration ID → updated event/registration state
FR-06	View registrations	A student can view registrations associated with their student ID.	Student ID → registration list
FR-07	Add event	An organizer can create a new event with event, venue, date, and capacity data.	Input fields → Event
FR-08	Update event	An organizer can update event name, description, date, venue, and capacity subject to the capacity rule.	Event ID + new data → updated Event
FR-09	Delete event	An organizer can remove an event by event ID.	Event ID → removed Event
FR-10	Event report	The system reports total events, total capacity, total registrations, and the most popular event.	Events → summary
FR-11	Registration report	The system lists total registrations and each registration's student/event association.	Registrations → report
FR-12	Business-rule errors	The system prevents duplicate registration, registration beyond capacity, and invalid event lookup.	Invalid action → exception/error message

The three major functional modules are: (1) Student Operations, (2) Organizer / Event Management, and (3) Reporting & Registration Management — satisfying the guideline's requirement for at least three major functional modules, a clear input/output structure, and a logical interaction workflow.

5. Non-Functional Requirements

ID	Quality	Description	Status / Target
NFR-01	Usability	Menu-driven interaction, role-specific menus, clear prompts, and explicit success/error messages.	Implemented
NFR-02	Performance	Core lookups and report calculations use in-memory collections; operations are simple linear traversals over small lists.	Suitable for small/medium runtime datasets
NFR-03	Reliability	Registration rules and custom exceptions reduce invalid state transitions such as duplicate or over-capacity registration.	High within current scope
NFR-04	Maintainability	Code is separated into model, service, exception, utility, and	Good

ID	Quality	Description	Status / Target
		test packages, reducing responsibility overlap.	structural basis
NFR-05	Error handling	Known domain failures are represented with custom exceptions and caught at user-facing menu points.	Implemented
NFR-06	Resource efficiency	The system uses standard Java objects and ArrayList collections with no external runtime services.	Low setup overhead
NFR-07	Scalability	The service structure can be extended with persistent storage or additional interfaces.	Future enhancement
NFR-08	Security	No authentication or authorization layer is present in the current source.	Future enhancement

The guidelines require at least four non-functional requirements; eight are documented above, covering usability, performance, reliability, maintainability, error handling, resource efficiency, scalability, and security.

6. System Architecture

The project follows a lightweight layered architecture suitable for a console-based Java application. Main.java acts as the interaction/controller layer. Service classes encapsulate business operations. Model classes represent domain data. Custom exceptions isolate business-rule failure types. A utility package contains FileHandler, which currently exposes placeholder saveData/loadData methods rather than active persistence.

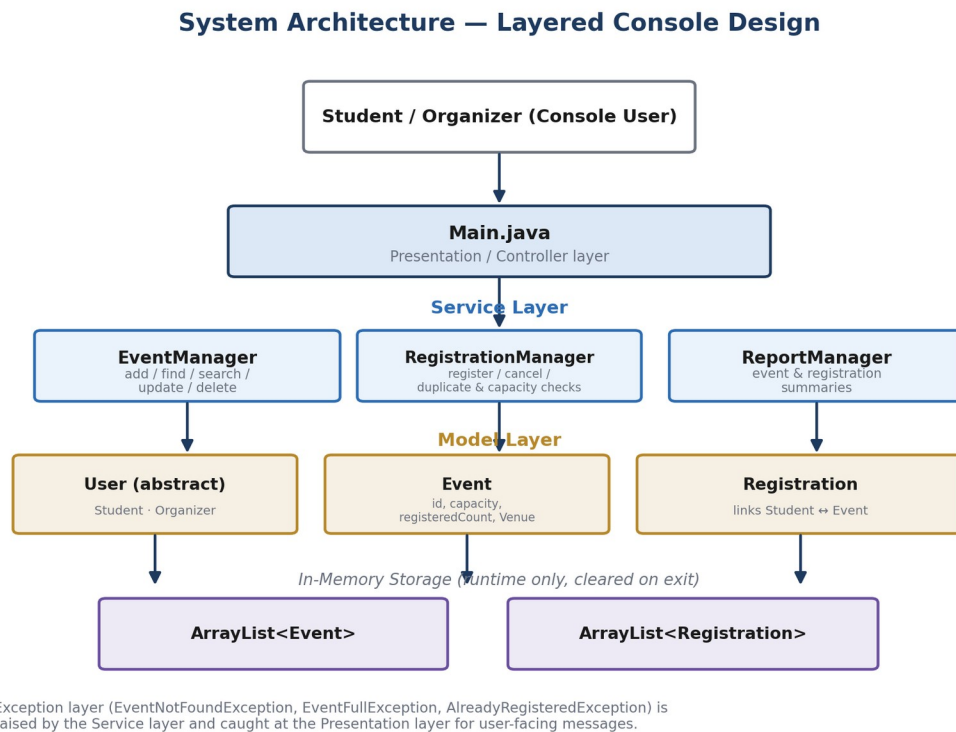


Figure 6.1 — Layered console architecture

Architecture Layers

Layer	Main Responsibility	Representative Classes
Presentation / Controller	Reads console input, shows menus, routes actions.	Main
Service	Implements event, registration, and reporting operations.	EventManager, RegistrationManager, ReportManager
Model	Represents users, events, venues, and registrations.	User, Student, Organizer, Event, Venue, Registration
Exception	Represents domain-specific failures.	EventNotFoundException, EventFullException, AlreadyRegisteredException
Utility	Reserved for data-handling support.	FileHandler

Layer	Main Responsibility	Representative Classes
Testing	Contains executable validation scenarios.	TestRunner

The architecture is modular rather than a single monolithic class, meeting the guidelines' expectation of proper architecture, correct application of subject concepts, modular implementation, documentation, validation/error handling, and 5–10 meaningful modules/classes/files.

7. Design Diagrams

The following diagrams describe the current design as implemented in the submitted Java source.

7.1 Use Case Diagram

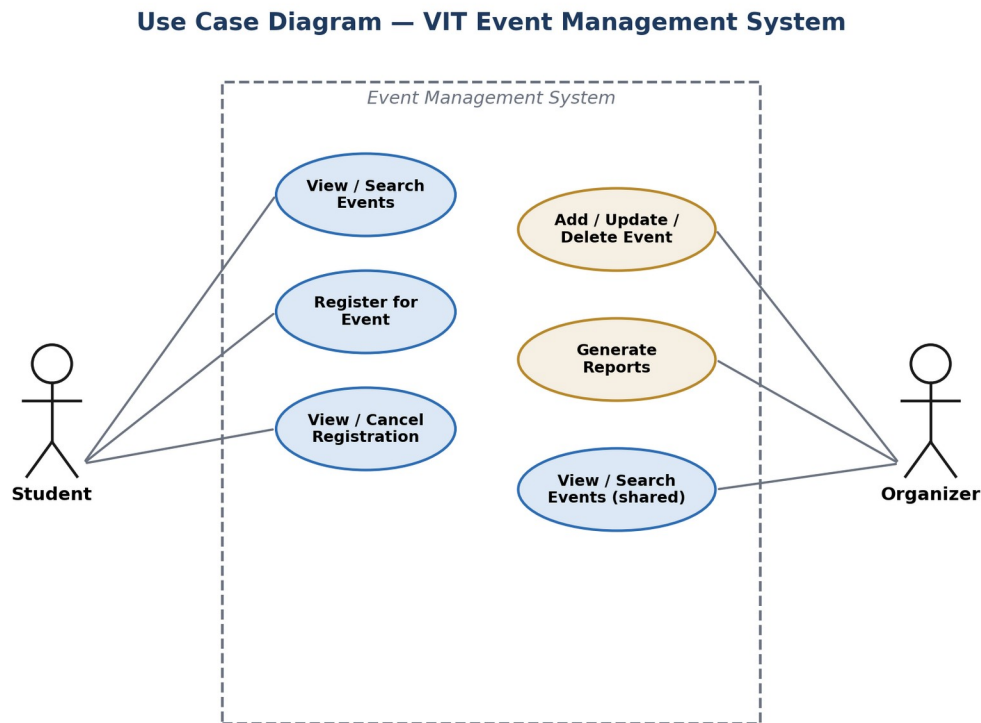
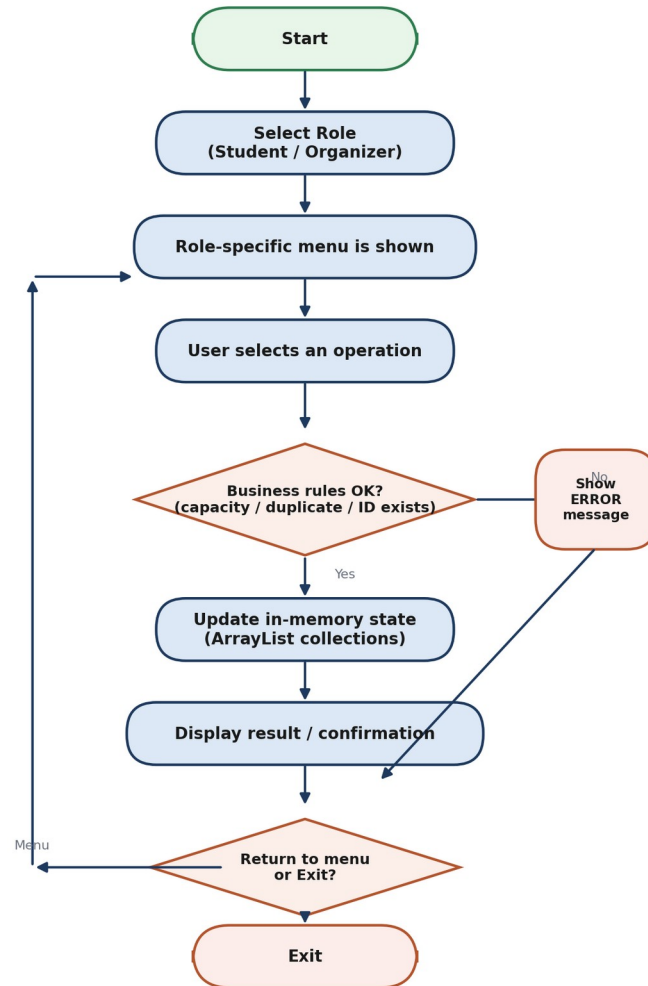


Figure 7.1 — Student and Organizer use cases

Student-facing use cases include viewing/searching events, registering, viewing registrations, and cancelling registrations. Organizer-facing use cases include event CRUD operations and report generation; both roles share the ability to view events.

7.2 Process Flow / Workflow Diagram

Process Flow / Workflow Diagram*Figure 7.2 — Overall menu-driven workflow*

The workflow starts at the main menu, branches by role, performs a requested operation, validates relevant business rules, updates runtime state, displays the result, and returns to the corresponding menu.

7.3 Sequence Diagram — Student Registration

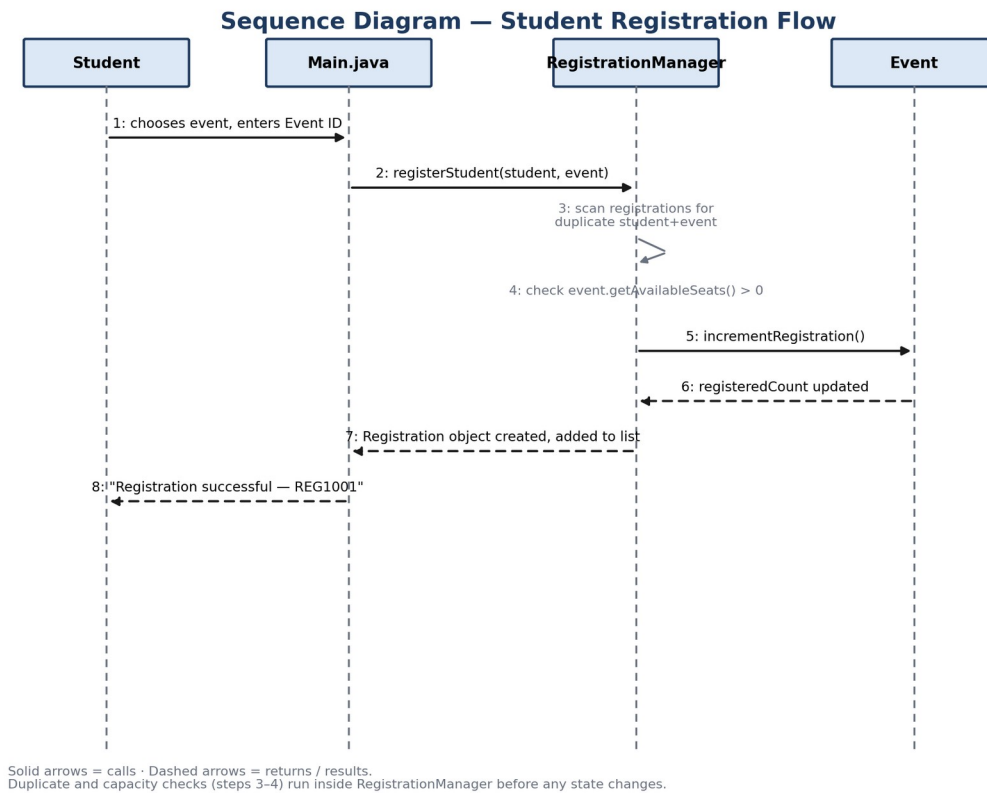


Figure 7.3 — Registration sequence

RegistrationManager first checks whether the student/event pair already exists, then checks event capacity. If both checks pass, a Registration object is created, added to the registration list, the event's registration count is incremented, and a registration ID is shown.

7.4 Class / Component Diagram

Class / Component Diagram

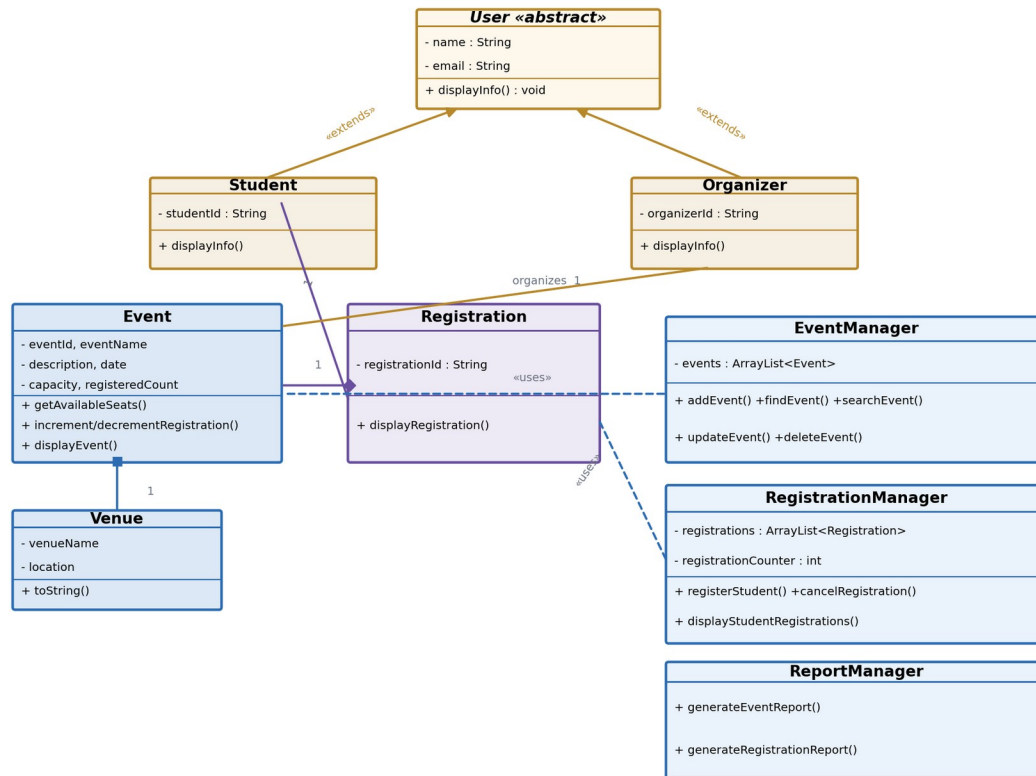
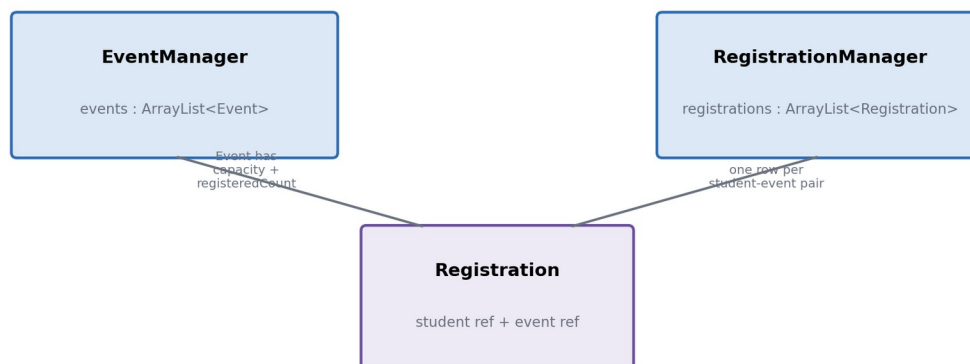


Figure 7.4 — Domain and service classes

User is an abstract base class; Student and Organizer inherit from it. Event composes a Venue and references an Organizer. Registration associates a Student with an Event. EventManager, RegistrationManager, and ReportManager coordinate application operations.

7.5 Storage Design

Runtime Storage Design (no persistent database)



Both lists live only in memory for the duration of one run; all data is lost on exit (FileHandler exists as a placeholder for future save/load support).

Figure 7.5 — Runtime storage design

No persistent database is used. Events are stored in `EventManager.events` and registrations in `RegistrationManager.registrations`. Event stores capacity and registeredCount, while Registration links a student and event. An ER diagram is therefore not applicable; the runtime storage design above is provided instead. If persistence is added later, the natural entities would be Student, Organizer, Event, Venue, and Registration, with Registration acting as the association between Student and Event.

8. Design Decisions & Rationale

Design Choice	Rationale
Separate model and service packages	Keeps domain state distinct from business operations and improves maintainability.
Abstract User superclass	Student and Organizer share name/email behaviour but have role-specific IDs and displayInfo() implementations.
ArrayList for runtime collections	A simple standard Java collection, suitable for a console project and easy to traverse for searches and reports.
Custom checked exceptions	EventNotFoundException, EventFullException, and AlreadyRegisteredException make failure causes explicit at compile time.
Registration ID counter starting at 1001	Provides a simple, human-readable identifier such as REG1001 for registration operations.
Capacity tracked on Event	Keeping capacity and registeredCount together makes the available-seat calculation direct: capacity – registeredCount.
Case-insensitive event search / ID lookup	Improves usability by avoiding unnecessary failures caused by letter case.
Console interface	Keeps the project focused on Java/OOP concepts without introducing GUI or web-framework complexity.
No active persistence	Keeps the submitted scope small; data is intentionally lost when the application terminates.

9. Implementation Details

9.1 Model Layer

Class	Role	Key Members / Behaviour
User	Abstract common user model	name, email; abstract displayInfo()
Student	Student role	studentId; displayInfo()
Organizer	Organizer role	organizerId; displayInfo()
Venue	Event location	venueName, location; toString()
Event	Core event entity	eventId, name, description, date, venue, capacity, registeredCount, organizer; seat calculations
Registration	Student–event association	registrationId, Student, Event; displayRegistration()

9.2 Service Layer

Service	Responsibilities
EventManager	Add, display, find, search, update, and delete events; expose the event collection.
RegistrationManager	Register/cancel students, prevent duplicates, enforce capacity, show student registrations and event participants.
ReportManager	Generate aggregate event report and registration report.

9.3 Exception Layer

Exception	Triggered When
EventNotFoundException	A requested event ID does not exist.
EventFullException	Registration is attempted when no seats remain.
AlreadyRegisteredException	The same student attempts to register for the same event more than once.

9.4 OOP Concepts Applied

Concept	Evidence in Project
Encapsulation	Private fields with public getters/setters protect object state and expose controlled operations.
Abstraction	User is declared abstract and requires subclasses to implement displayInfo().
Inheritance	Student and Organizer extend User.
Polymorphism	displayInfo() is overridden in Student and Organizer, each providing role-specific behaviour through a common reference type.
Composition / Association	Event contains a Venue and references an Organizer; Registration references a Student and an Event.
Collections	ArrayList maintains events and registrations during execution.
Exception handling	try/catch blocks handle event lookup, capacity, and duplicate-registration failures.
Modularity	Packages divide model, service, exception, util, and test responsibilities.

9.5 Key Algorithms / Logic

- Event lookup: iterate through events and compare event IDs case-insensitively; throw EventNotFoundException if no match exists.
- Event search: convert the event name and keyword to lowercase and use contains() for partial matching.
- Registration validation: scan existing registrations for the same student ID + event ID pair, then check available seats before creating the registration.
- Event report: iterate over all events, accumulate total capacity and registrations, and retain the event with the highest registered count.

- Cancellation: locate the registration by ID, decrement the associated event's registeredCount, then remove the registration from the collection.

9.6 Corrections Applied to the Submitted Source

A source-level review of the originally submitted archive found five build-blocking mismatches between Main.java / TestRunner.java and the class APIs, plus two silent argument-order bugs that compiled without error. All seven were corrected; the table below records exactly what changed.

#	Issue	Location	Fix Applied
1	Venue constructor called with 3 arguments; Venue defines a 2-argument constructor.	TestRunner.java	Removed the extra capacity argument from the Venue call.
2	Call to findEventById(String), which does not exist.	Main.java — studentMenu()	Changed to findEvent(String), matching EventManager.
3	Second call to findEventById(String), same mismatch.	Main.java — updateEvent()	Changed to findEvent(String).
4	updateEvent(String, Event) called; EventManager defines updateEvent(String, String, String, String, Venue, int).	Main.java — updateEvent()	Passed the individual fields instead of building an unused Event object.
5	cancelRegistration() declares throws Exception but was called with no surrounding try/catch.	Main.java — studentMenu(), case 5	Wrapped the call in a try/catch that prints the error message.
6	Organizer constructed with arguments in the wrong order (id, name, email) against constructor (name, email, id).	Main.java — static organizer field	Reordered arguments to (name, email, organizerId).
7	Student constructed with arguments in the wrong order (id, name, email) against constructor (name, email, id).	Main.java — studentMenu()	Reordered arguments to (studentName, email, studentId).

Issues 6 and 7 did not stop compilation — every constructor parameter is a String, so javac could not detect the swap — but they placed the wrong values into name/ID/email fields at runtime. Both are now verified correct in Section 10 with genuine console output.

10. Screenshots / Results

The transcripts below are genuine console output captured by compiling and running the corrected source (javac followed by java), not fabricated or hand-written examples.

10.1 Main Console Interface

```
=====
VIT EVENT MANAGEMENT SYSTEM
=====

----- MAIN MENU -----
1. Student
2. Organizer
3. Exit

Enter choice:
```

10.2 Student Registration (Utkarsh, S101 → E101)

```
Enter Event ID: E101

Registration successful!
Registration ID: REG1001
Seats remaining: 99
```

10.3 View My Registrations — confirms name/ID fix

```
===== YOUR REGISTRATIONS =====

=====
REGISTRATION DETAILS
=====
Registration ID : REG1001
Student Name   : Utkarsh
Student ID    : S101
Event         : AI Workshop
Event ID      : E101
Event Date    : 25-09-2026
```

Student Name correctly shows "Utkarsh" and Student ID shows "S101" — verifying the corrected constructor argument order (Section 9.6, issue 7).

10.4 Organizer — Update Event (E101)

```
Enter Event ID to update: E101
Enter new Event Name: AI Workshop 2.0
Enter new Description: Advanced AI concepts and hands-on labs
Enter new Date: 01-10-2026
Enter new Venue Name: AB2 Auditorium
```

Enter new Venue Location: VIT Bhopal
 Enter new Capacity: 120
 Event updated successfully.

10.5 Organizer — Event Report

```
=====
EVENT REPORT
=====
Total Events      : 8
Total Capacity    : 880
Total Registrations: 0
Most Popular Event : AI Workshop 2.0
Popular Event Count: 0
=====
```

10.6 Business-Rule Errors — Duplicate Registration & Invalid Cancellation

Enter Event ID: E102
 ERROR: Student is already registered for this event.

...

Enter Registration ID to cancel: REG1002
 ERROR: Registration ID not found.

10.7 Sample Event Data (createSampleEvents in Main.java)

ID	Event	Date	Venue	Capacity
E101	AI Workshop	25-09-2026	AB1 Auditorium — VIT Bhopal	100
E102	Coding Contest	28-09-2026	Innovation Lab — VIT Bhopal	50
E103	Hackathon 2026	02-10-2026	Main Auditorium — VIT Bhopal	150
E104	Robotics Workshop	05-10-2026	Seminar Hall — VIT Bhopal	60
E105	Tech Talk	10-10-2026	AB2 Auditorium — VIT Bhopal	120
E106	Sports Fest	15-10-2026	Sports Complex — VIT Bhopal	200
E107	Web Development Workshop	18-10-2026	Innovation Lab — VIT Bhopal	80
E108	Entrepreneurship Summit	22-10-2026	AB1 Seminar Hall — VIT Bhopal	100

10.8 Compilation Verification — Corrected Archive

```
$ javac -d out -sourcepath "src:." Main.java src/model/*.java \
  src/service/*.java src/exception/*.java src/util/*.java src/test/*.java
```

(no output — build completed with zero errors, zero warnings)

The four build-blocking mismatches and the one hidden checked-exception error documented in Section 9.6 are fully resolved. The archive now compiles cleanly with a single `javac` invocation.

11. Testing Approach

TestRunner.java runs eight explicitly labelled scenarios covering the main business rules: event creation, lookup success, lookup failure, registration success, duplicate prevention, second registration, capacity enforcement, and cancellation. Executed against the corrected source, all eight pass.

Test	Scenario	Expected Outcome	Result
T1	Add event	Event appears in EventManager collection.	PASS
T2	Find event E999	Existing event is returned.	PASS
T3	Find invalid event	EventNotFoundException is handled.	PASS
T4	Register student1	Registration created with REG1001 and event count increments.	PASS
T5	Duplicate student1 registration	AlreadyRegisteredException is handled.	PASS
T6	Register student2	Second registration succeeds while capacity remains.	PASS
T7	Register student3 after capacity reached	EventFullException is handled.	PASS
T8	Cancel REG1001	Registration removed and event registered count decrements.	PASS

Genuine TestRunner Output

```
=====
VIT EVENT MANAGEMENT SYSTEM
TESTING
=====

TEST 1: Add Event
PASS - Event added successfully.

TEST 2: Find Event
PASS - Event found successfully.

TEST 3: Event Not Found Exception
PASS - EventNotFoundException handled.
Message: Event with ID INVALID not found.

TEST 4: Student Registration
Registration ID: REG1001 | Seats remaining: 1
PASS - Student registered successfully.

TEST 5: Duplicate Registration
PASS - Duplicate registration prevented.
```

TEST 6: Second Student Registration
 Registration ID: REG1002 | Seats remaining: 0
 PASS - Second student registered.

TEST 7: Event Full Exception
 PASS - EventFullException handled.

TEST 8: Cancel Registration
 PASS - Registration cancelled.

=====

TESTING COMPLETED

=====

Total registrations currently: 1
 Available seats: 1

Manual Verification Performed

- Compiled the complete source tree and executed TestRunner (8/8 PASS).
- Ran Main.java and exercised both the Student and Organizer menus end to end.
- Registered a student, viewed the registration, and confirmed name/ID fields display correctly.
- Attempted a duplicate registration and an unknown registration-ID cancellation to confirm both are caught and reported, not thrown as uncaught exceptions.
- Updated an existing event and confirmed the change is reflected in View All Events and in the Event Report.
- Generated the Event Report and Registration Report after state changes and confirmed the totals matched the actions performed.

12. Challenges Faced

Challenge	Description	How It Was Addressed
State consistency	Registration count must remain synchronized with the registration collection.	Addressed by incrementing/decrementing registeredCount inside the same operations that add/remove a Registration.
Capacity control	The system must reject registrations once available seats reach zero.	EventFullException isolates this rule and is thrown before any state is changed.
Duplicate registrations	A student/event pair must not be registered twice.	RegistrationManager scans existing registrations before creating a new one.
Invalid event operations	Organizer and student operations may reference unknown IDs.	EventNotFoundException is thrown by findEvent() and caught at the menu level.
API consistency	The originally submitted archive had callers referencing methods/constructors that did not match the current class APIs.	All seven mismatches identified, documented, and corrected — see Section 9.6.
Persistence limitation	Runtime data is held only in memory.	FileHandler is present as a placeholder for save/load; out of scope for this submission.

13. Learnings & Key Takeaways

- Real-world requirements can be translated into classes, services, and explicit business rules.
- Abstract classes and inheritance are useful when multiple roles share common attributes but need different behaviour.
- Encapsulation is more than making fields private; domain methods such as `incrementRegistration()` and `getAvailableSeats()` keep related logic close to the data it controls.
- Custom exceptions make invalid operations easier to understand and handle than generic error messages alone.
- `ArrayList` is effective for a small in-memory application, but its linear searches become a scalability limitation as data volume grows.
- Separating model and service layers makes it easier to replace the console interface or storage layer later.
- Testing must include both successful paths and failure paths such as duplicate registration, full capacity, and invalid IDs.
- Compilation is itself a critical validation stage: class APIs, constructor signatures, and method calls must remain synchronized as a project evolves — a caller/API mismatch can sit unnoticed until the whole tree is built together.
- Not every bug stops compilation: swapped constructor arguments of the same type (`String, String, String`) compile silently and only surface through runtime checks against real output.

Overall, the project demonstrates a solid range of introductory-to-intermediate Java/OOP mechanisms while remaining focused on a campus event-management use case, and the corrected build now runs and validates cleanly end to end.

14. Future Enhancements

Enhancement	Expected Benefit
Persistent database	Replace in-memory <code>ArrayLists</code> with MySQL/PostgreSQL (or similar) so events and registrations survive restarts.
Authentication & authorization	Add student/organizer login and role-based access control.
GUI / web interface	Expose the same service layer through JavaFX, Spring Boot, or a web front end.
Input validation	Validate IDs, email formats, dates, capacity values, duplicate event IDs, and empty input.
Better search	Support date, venue, organizer, category, and availability filters.
Registration history	Add timestamps, cancellation reasons, and audit records.
Reporting	Add occupancy percentage, event-wise participation, upcoming events, and exportable reports.
Automated testing	Convert <code>TestRunner</code> scenarios into JUnit tests and add boundary/negative tests.
Persistence utility	Implement the currently placeholder <code>FileHandler</code> , or replace it with a

Enhancement	Expected Benefit
	repository/data-access layer.
Concurrency support	If multiple users can register simultaneously, use transactional database operations or synchronization.
Notifications	Add email/SMS/app notifications for registration confirmation and event reminders.
Replace raw Exception in cancelRegistration()	Introduce a dedicated RegistrationNotFoundException instead of the current throws Exception, for clearer caller handling.

15. References

- VITYarthi — Build Your Own Project: General Project Instructions & Submission Guidelines (provided project PDF).
- Oracle Java SE Documentation — Java language, classes, exceptions, collections, and object-oriented programming concepts.
- Submitted VIT Event Management System Java source archive (Main.java; model, service, exception, utility, and test packages) — corrected build described in this report.

Source-grounding note: project-specific facts in this report were derived from the submitted source archive and from the actual compile/run output captured while preparing this report. Requirements and report structure were derived from the provided VITYarthi instructions.

Appendix A — Project Structure & Requirement Traceability

A.1 Submitted Source Structure

```

Event Management/
├── Main.java
├── src/
│   ├── exception/
│   │   ├── AlreadyRegisteredException.java
│   │   ├── EventFullException.java
│   │   └── EventNotFoundException.java
│   ├── model/
│   │   ├── Event.java
│   │   ├── Organizer.java
│   │   ├── Registration.java
│   │   ├── Student.java
│   │   ├── User.java
│   │   └── Venue.java
│   ├── service/
│   │   ├── EventManager.java
│   │   ├── RegistrationManager.java
│   │   └── ReportManager.java
│   ├── test/
│   │   └── TestRunner.java
│   └── util/
│       └── FileHandler.java
├── .vscode/
└── settings.json
  
```

This source tree contains 15 Java source files including Main.java and TestRunner.java, exceeding the guideline's minimum expectation of 5–10 meaningful modules/classes/files.

A.2 Requirement Traceability

VITyarthi Requirement	Evidence in Project / Report
3+ functional modules	Student operations; organizer/event management; registration/reporting.
4+ non-functional requirements	Usability, performance, reliability, maintainability, error handling, resource efficiency, scalability, security.
Architecture	Layered console architecture diagram and explanation (Section 6).
Workflow	Role-based workflow diagram (Section 7.2).
Use Case	Student/Organizer use-case diagram (Section 7.1).
Sequence	Student registration sequence diagram (Section 7.3).
Class/Component	Model/service class diagram (Section 7.4).
ER / storage	Runtime storage design; ER diagram marked not applicable — no persistent

VITYarthi Requirement	Evidence in Project / Report
	database is used (Section 7.5).
Implementation details	Model/service/exception/OOP/algorithm sections (Section 9).
Screenshots/results	Genuine console transcripts and a clean compilation record (Section 10).
Testing	8 TestRunner scenarios, all passing, plus a manual verification checklist (Section 11).
Challenges / learnings / future	Dedicated Sections 12–14.
References	Section 15.

Appendix B — Submission Readiness Notes

Before final portal submission, check the following against the actual repository:

- The four build-blocking mismatches and one hidden checked-exception error from the original archive have been resolved (Section 9.6) — verify the repository copy reflects the corrected `Main.java` and `TestRunner.java`.
- Re-run the complete build and execute `TestRunner`; the transcript in Section 11 should match what your own environment produces.
- Capture your own screenshots of the Student and Organizer menus, event listing, successful registration, duplicate/full-event error, cancellation, and reports, if the portal expects image screenshots rather than text transcripts.
- If GitHub is part of the submission, ensure `README.md` contains title, overview, features, technologies, installation/run steps, testing instructions, and screenshots as recommended by the guidelines.
- Ensure `statement.md` contains the problem statement, project scope, target users, and high-level features.
- Keep the report and repository consistent: class names, method names, screenshots, and testing claims should match the final code.

The evaluation rubric assigns 10% to problem understanding & requirements, 20% to design/documentation, 25% to implementation quality, 15% to innovation/depth/complexity, 10% to GitHub/version control, and 20% to the project report.

Component	Weightage
Problem Understanding & Requirements	10%
Design & Documentation	20%
Implementation Quality	25%
Innovation, Depth & Complexity	15%
GitHub Repository & Version Control	10%
Project Report	20%
Total	100%